

Membership Renewal System

Author: isaasi

Last Updated: February 2026

Module: Organisation Membership

Table of Contents

1. Overview
 2. Architecture
 3. The Simulation Engine
 4. Cost Calculation Logic
 5. Override System
 6. Invoicing Modes
 7. The Three Renewal Paths
 8. Record Fee
 9. Safeguards and Duplicate Prevention
 10. Frontend UI — Organisation Membership Tab
 11. Purchase Order Numbers
 12. Email Fees & Member Payment Portal
 13. Portal Membership Fees Page
 14. Membership Settings
 15. Fee Approval Workflow
 16. Database Tables
 17. Data Flow Diagrams
 18. Xero Integration
 19. Configuration Reference
 20. Troubleshooting
-

Overview

The membership renewal system handles the full lifecycle of calculating, creating, and invoicing annual membership fees for organisations within a tenant. It is designed around a single source of truth — the shared simulation function — that all three renewal paths (workflow, cron job, manual) use to ensure consistent cost calculations everywhere.

Each organisation's membership year is determined by the tenant's tier configuration (configurable start month/day), and costs are calculated using a day-based approach that accounts for pro-rata periods, free periods, spillover discounts, custom discount rules, and three types of overrides.

The system supports per-year invoicing controls, allowing admins to independently choose Automatic, Scheduled, or Manual invoicing for each membership year. An optional fee approval workflow allows admins to review and approve calculated fees per organisation/year before any processing (cron, manual renewal, or member payment) can proceed.

Architecture

Key Files

File	Purpose
api/_lib/membershipSimulation.js	Shared simulation engine — single source of truth for all cost calculations
api/_lib/membershipConfigResolver.js	Resolves the correct tier configuration for an organisation (handles scoped configs)
api/_lib/discountHelper.js	Evaluates and applies custom discount rules
api/_lib/workflows.js	Workflow <code>create_membership</code> action (automatic path)
api/cron/process-membership-renewals.js	Cron job for automatic and scheduled renewals
api/membership/org-membership-invoicing.js	Manual renewal endpoint and invoicing settings management
api/membership/member-fees.js	Authenticated portal API — session-based fee lookup, PO submission, Stripe payment
api/membership/email-fees.js	Email Fees endpoint — token creation, email sending
api/public/membership-fees/[token]	Public API — token validation, PO submission, Stripe payment
api/membership/membership-settings.js	Membership settings API — approval toggle, Stripe enablement, custom message
client/src/components/OrgMembershipForm.js	React.js UI for the organisation membership tab
client/src/pages/MembershipSettingsAdmin	Admin page for membership settings (approval, Stripe, custom message)
client/src/pages/MembershipFeePagePublic	Public member-facing payment page (token-based)
client/src/pages/MembershipFees.js	Portal membership fees page (authenticated members)

Design Principles

1. **Single source of truth:** `simulateMembershipForOrg()` is the only function that performs cost calculations. No renewal path implements its own calculation logic.
2. **Day-based calculations:** Pro-rata and billable-day calculations use actual calendar days between dates. Free period durations are first converted from their configured unit (months, weeks, or days) into an approximate day count using standard conversion factors (months $\times$ 30.44, weeks $\times$ 7), then applied as calendar days within the membership year.
3. **Per-year independence:** Each membership year has its own invoicing mode, override, and simulation — they don't interfere with each other.
4. **Tenant isolation:** All queries are scoped by `tenant_id` to ensure strict multi-tenant data separation.
5. **Defensive coding:** Three layers of duplicate prevention ensure a membership record is never created twice.
6. **Optional approval gate:** When enabled, fee approval adds a consistent check across all four processing paths (cron, manual, portal, public token) using the same pattern — query the tenant setting, then check the per-org/year flag.

The Simulation Engine

Function Signature

`simulateMembershipForOrg(tenantId, organizationId, options = {})`

Options

Option	Type	Default	Description
<code>source</code>	string	<code>'workflow'</code>	Who is calling: <code>'workflow'</code> , <code>'cron'</code> , <code>'manual'</code> , or <code>'simulate'</code>
<code>mode</code>	string	<code>'automatic'</code>	Invoicing mode context: <code>'automatic'</code> , <code>'scheduled'</code> , or <code>'manual'</code>
<code>workflowName</code>	string	<code>null</code>	Name of the triggering workflow (for logging)
<code>verbose</code>	boolean	<code>false</code>	Enable extra logging
<code>targetYear</code>	string	<code>null</code>	Target a specific year label (e.g. <code>'2025/2026'</code>). If null, defaults based on source

Year Targeting Logic

When no `targetYear` is specified: - `source === 'simulate'` → targets the **next** membership year (preview) - All other sources → target the **current** membership year

When `targetYear` is specified, the simulation matches it against the current or next year labels and uses the matching one.

Return Value

The simulation returns a comprehensive result object containing:

- `success` (boolean) — whether simulation completed
- `org` — the organisation record
- `config` — the matched tier configuration
- `membershipYear` — { `label`, `start`, `end` } for the target year
- `yearNumber` — which year of membership this is (1st, 2nd, 3rd, etc.)
- `goLiveDate` — the organisation's go-live date (or null)
- `matchedBand` — the tier band that was matched
- `tierLabel` — display label for the matched tier
- `fieldValue` — the organisation's field value used for band matching
- `annualCost` — the base annual cost (after custom discounts but before pro-rata/free period)
- `prorataCost` — pro-rata cost if applicable (year 1 only)
- `freeDiscount` — free period discount amount
- `rolloverDiscount` — rollover discount amount (year 2 spillover)
- `customDiscountTotal` — total of all custom discount rules
- `customDiscountDetails` — array of applied discount rule details

- `finalCost` — the actual amount to charge
- `currency` — currency code (e.g. 'GBP')
- `overrideApplied` — whether an override was used
- `overrideType` — type of override if applied
- `existingRecord` — existing history record if one exists for this year (includes `xero_invoice_id`)
- `invoicePreview` — preview of the Xero invoice that would be generated
- `steps` — detailed step-by-step log of the simulation (used in UI)
- `isNewOrg` — whether this is a new organisation (year 1 with no existing records)
- `billingPeriod` — from config (typically 'annual')

Simulation Steps Log

The simulation builds a detailed `steps` array as it progresses, recording each decision point. This is displayed in the frontend simulation panel and is invaluable for debugging. Each step has:

```
{ step: 'Step Name', detail: 'Description', status: 'ok' | 'warning' | 'error', timestamp: '..
```

Cost Calculation Logic

Membership Year Determination

The membership year is defined by the tier configuration's `membership_start_month` and `membership_start_day`. For example, if the membership year starts on April 1:

- Current year: 2025/2026 (1 Apr 2025 – 31 Mar 2026)
- Next year: 2026/2027 (1 Apr 2026 – 31 Mar 2027)

The label format is always YYYY/YYYY.

Year Number Calculation

The year number tells us how long an organisation has been a member, counted from their go-live date:

- **Year 1:** The membership year containing the go-live date
- **Year 2:** The next membership year after year 1
- **Year 3+:** Subsequent years

This is important because pro-rata and free period discounts only apply in years 1 and 2.

Tier Band Matching

1. The system looks up the organisation's field value (e.g. member count, or a custom field)
2. It matches this value against the tier bands defined in the configuration
3. Each band has a `min_value`, `max_value` (null = unlimited), `annual_cost`, `label`, and optionally `vat_rate`
4. The first band where `fieldValue >= min_value AND fieldValue <= max_value` is selected

Custom Discounts

Before any time-based calculations, custom discount rules are evaluated:

1. Discount rules are defined per tier configuration
2. Each rule checks an organisation's custom field value against conditions
3. Rules can apply percentage or fixed-amount discounts
4. Multiple rules stack (applied sequentially)
5. The discounted cost becomes the basis for all subsequent calculations

Day-Based Calculations (Year 1)

When an organisation is in their first membership year and pro-rata is enabled:

```
totalDaysInYear = days from year start to year end (inclusive)
dailyCost = annualCost / totalDaysInYear
prorataDays = days from go-live date to year end (inclusive)
prorataCost = dailyCost × prorataDays
```

Free Period (Year 1)

If the configuration has a free period:

```
freePeriodTotalDays = free period converted to days
freePeriodDaysApplied = min(freePeriodTotalDays, prorataDays) // can't exceed remaining days
freeDiscount = dailyCost × freePeriodDaysApplied
```

Final Cost (Year 1)

```
billableDays = prorataDays - freePeriodDaysApplied
finalCost = dailyCost × billableDays
```

Free Period Spillover (Year 2)

If the free period wasn't fully consumed in year 1 (because the org joined late in the year), the remaining free days spill over into year 2:

```
spilloverDays = freePeriodTotalDays - freeDaysUsedInYear1
year2FreeDiscount = year2DailyCost × min(spilloverDays, totalDaysInYear2)
finalCost = annualCost - year2FreeDiscount
```

Year 3+

No pro-rata, free period, or spillover discounts. The full annual cost applies (after custom discounts and any overrides).

Price Override

When a price override is set, all calculation lines (pro-rata, free period, spillover) are suppressed. The final cost is exactly the manual price specified.

Override System

Overrides allow admins to modify an organisation's membership cost outside the normal tier calculation. There are three types:

1. Structure Override

Assigns the organisation to a different tier configuration entirely.

- **Config ID:** Points to an alternative `membership_tier_config`
- **Band ID** (optional): Forces a specific band within that config. If not set, the organisation's field value is matched against the alternative config's bands
- Custom discounts are re-evaluated against the new config
- Pro-rata and free period calculations still apply based on the new annual cost

2. Manual Price Override

Sets an exact price, bypassing all calculations.

- The `manual_price` value becomes the final cost directly
- No pro-rata, free period, custom discounts, or spillover are applied
- Useful for negotiated or exceptional arrangements

3. Discount Override

Applies a specific discount instead of the normal custom discount rules.

- **Discount type:** 'percentage' or 'fixed'
- **Discount value:** The percentage or fixed amount
- When applied, it replaces the custom discount rules entirely (not stacked on top)
- The gross cost is recalculated as `annualCost + customDiscountTotal` (i.e. the cost before custom discounts were applied), and the override discount is applied against that gross cost
- For percentage: `discountAmount = grossCost × percentage / 100`
- For fixed: `discountAmount = min(fixedValue, grossCost)`

Override Scoping

Overrides are stored in the `organisation_membership_override` table with an optional `membership_year` column:

- Year-specific overrides take priority (matched by year label)
- Legacy overrides (null `membership_year`) serve as fallback
- The simulation queries using `OR(membership_year = targetYear, membership_year IS NULL)` and picks the most specific match

Invoicing Modes

Each organisation can have a different invoicing mode per membership year, stored in `organisation_membership_invoicing`.

Automatic

- **What happens:** The membership record and invoice are created together. Typically, the cron job is the primary actor — it creates both the record and invoice in one pass. If a workflow fires first (e.g. when go-live is set), it creates just the record; the cron then finds the existing record and skips.
- **Workflow action:** Proceeds normally — creates the membership history record only (no Xero invoice).
- **Cron job:** Handles automatic renewals — checks if the membership year has started and creates records with invoices for eligible orgs. If the workflow already created the record, the cron will find the existing record and skip entirely (in automatic mode, the cron does not separately invoice existing records — invoicing only happens when the cron creates the record itself).

Scheduled

- **What happens:** The membership record is created when the year starts, but the invoice is generated on a specific future date chosen by the admin.
- **Workflow action:** Skips — returns a message saying the scheduled renewal job will handle it.
- **Cron job:** Creates the record when the year starts (without invoicing). On a subsequent run, when the invoice date arrives, it generates the Xero invoice for the existing record.
- **VAT handling:** When the cron job invoices an existing record later, it derives VAT from the stored record's `vat_rate` or falls back to looking up the band's VAT rate — it does NOT re-run the simulation, ensuring the invoiced amount matches the recorded amount.

Manual

- **What happens:** Nothing automatic — the admin must click the “Renew & Invoice Now” button in the UI to create both the record and invoice.
- **Workflow action:** Skips — returns a message directing to the admin UI.
- **Cron job:** Ignores organisations with manual mode (only queries for `automatic` and `scheduled`).
- **UI:** The “Renew & Invoice Now” button only appears on the current year section, not the next year preview.

Default Behaviour

If no invoicing setting exists for an organisation/year: - The **workflow action** defaults to 'automatic' (so it will proceed to create the record) - The **simulation engine** logs the saved mode from the invoicing table; if no setting exists it reports 'manual' as the current mode for display context - The **cron job** won't pick it up — it only queries `organisation_membership_invoicing` rows that explicitly have `automatic` or `scheduled` mode, so orgs with no saved setting are naturally excluded

Legacy Fallback

The `membership_year` column in `organisation_membership_invoicing` was added later. Legacy rows with null `membership_year` serve as fallback when no year-specific setting exists.

The Three Renewal Paths

Path 1: Workflow Action (`create_membership`)

File: `api/_lib/workflows.js` → `executeCreateMembershipAction()`

Trigger: A workflow containing a `create_membership` action is executed (typically triggered by setting a go-live date on an organisation).

Flow:

1. Resolve the organisation ID (supports both organisation and member entity types)
2. Call `simulateMembershipForOrg()` with `source: 'workflow'`
3. If simulation fails → return error with steps
4. If dry run → return simulation results without creating anything
5. **Check invoicing mode** for the target year (with null fallback):
 - `manual` → skip with message to use admin UI
 - `scheduled` → skip with message about cron job
 - `automatic` (or no setting) → proceed
6. If existing record found → skip (duplicate prevention)
7. Insert membership history record
8. Return success with full cost breakdown

Important: The workflow action only creates the membership history record. It does NOT generate a Xero invoice. Invoicing is handled separately by the cron job (for automatic/scheduled modes) or the manual renewal path.

Path 2: Cron Job (Automatic & Scheduled)

File: `api/cron/process-membership-renewals.js`

Trigger: Scheduled cron execution (e.g. daily).

Flow:

1. Fetch all active tier configurations to determine which tenants to process
2. For each tenant, query `organisation_membership_invoicing` for rows with `automatic` or `scheduled` mode
3. For each org/setting:
 - a. Call `simulateMembershipForOrg()` with `source: 'cron'` and the setting's `membership_year` as `targetYear`
 - b. **Go-live date guard:** If no go-live date → skip
 - c. Determine if renewal is due (today ≥ year start date)

Automatic mode flow: - If not due → skip - If existing record → skip - Create record AND invoice together

Scheduled mode flow: - If not due AND no existing record → skip - If no existing record AND due → create record (invoice only if invoice date reached) - If existing record WITHOUT invoice AND invoice date reached → invoice the existing record - If existing record WITH invoice → skip

4. Log results to `scheduled_task_log`

Key details: - Uses `simulateMembershipForOrg()` for all cost calculations - Three-layer duplicate prevention (see Safeguards section) - Scheduled invoicing of existing records derives VAT from the stored record, not from re-simulation

Path 3: Manual Renewal

File: `api/membership/org-membership-invoicing.js` → `handleManualRenewal()`

Trigger: Admin clicks “Renew & Invoice Now” button in the UI (current year section only).

Flow:

1. Accept `organizationId` and optional `membershipYear` from request body
2. Call `simulateMembershipForOrg()` with `source: 'manual', mode: 'manual',` and `targetYear` (defaults to current year)
3. **Go-live date guard:** If no go-live date → return 400 error
4. **Duplicate guard:** If existing record → return 400 error
5. Insert membership history record
6. **DB constraint guard:** Handle 23505 (unique violation) gracefully
7. Create Xero invoice immediately
8. Link invoice to history record
9. Create organisation note with renewal and invoice details
10. Return success response

Key detail: Manual renewal creates BOTH the record AND invoice in one operation, giving the admin immediate results.

Record Fee

Purpose

The Record Fee button allows an admin to manually capture the current year’s calculated membership cost as a history record — **without generating a Xero invoice**. This is distinct from the three renewal paths described above, which are all part of the invoicing lifecycle.

How It Works

1. Admin clicks “Record {year} Fee” button on the current year card
2. The frontend sends a POST to `/api/membership/org-membership` with `{ organizationId, membershipYear }`
3. The backend runs the simulation to calculate the current cost (including any active overrides)
4. The simulation result’s **final cost** is saved as a `organisation_membership_history` record
5. No Xero invoice is generated

What Gets Recorded

The recorded fee is the **final cost shown on the year card** at the time the button is clicked. This means:

- If a **structure override** is active, the recorded cost reflects the alternative tier/band pricing

- If a **manual price override** is set, the recorded cost is exactly that manual price
- If a **discount override** is active, the recorded cost reflects the overridden discount
- If **no override** is active, the recorded cost reflects the standard tier calculation with any custom discounts, pro-rata, and free period adjustments

Record Fee vs Renew & Invoice Now

Aspect	Record Fee	Renew & Invoice Now
Creates history record	Yes	Yes
Generates Xero invoice	No	Yes
Appears on	Current year card only	Current year card only (when mode is Manual)
Use case	Lock in the calculated cost without invoicing	Create record and send invoice immediately

UI Behaviour After Recording

Once a fee is recorded for the current year:

- The **Record Fee** button is replaced by a badge showing “Recorded: {amount}”
- The **Final Cost** label changes to “Recorded Cost”
- The **entire invoicing section is hidden** (radio buttons, date picker, Save button, and Renew & Invoice Now button) — replaced by a message: “Fee recorded for {year} — invoicing controls hidden”
- The **next year card is unaffected** — it continues to show all invoicing controls as normal

When to Use

- When you want to lock in the cost calculation for the year before deciding how/when to invoice
- When invoicing will be handled outside the platform (e.g. manually through Xero directly)
- When you need a record of the agreed cost but don’t want to trigger an automatic invoice

Safeguards and Duplicate Prevention

We implement three layers of duplicate prevention to ensure a membership record is never created twice for the same year:

Layer 1: Simulation Check (Application Level)

The simulation function queries for existing records:

```
const { data: existingRecord } = await supabase
  .from('organisation_membership_history')
  .select('id, membership_year, final_cost, xero_invoice_id')
```

```

.eq('tenant_id', tenantId)
.eq('organization_id', organizationId)
.eq('membership_year', membershipYear.label)
.maybeSingle();

```

All callers check `simResult.existingRecord` and skip if present.

Layer 2: Caller Re-check

Each renewal path has its own duplicate check before insertion: - **Workflow:** Checks `simResult.existingRecord` after invoicing mode check - **Cron:** `processOrgRenewal()` re-checks `simResult.existingRecord` at the start - **Manual:** Checks `simResult.existingRecord` before insertion

Layer 3: Database Constraint

If a race condition somehow gets past both application-level checks, the database unique constraint on `(tenant_id, organization_id, membership_year)` in `organisation_membership_history` catches the duplicate. The error code 23505 is handled gracefully:

```

if (insertError.code === '23505') {
  // Return "skipped" or "already exists" instead of throwing
}

```

Go-Live Date Guard

All automated renewal paths require a go-live date: - **Cron:** Skips orgs without a go-live date with a clear log message - **Manual:** Returns a 400 error explaining a go-live date is required - **Workflow:** The simulation still runs (to allow dry runs/previews) but the go-live date is reported. The workflow action itself only proceeds for automatic mode, which typically is triggered by setting the go-live date

Fee Approval Guard

When `membership_require_approval` is enabled in `system_settings`, all processing paths check the `fees_approved` flag on the org/year's invoicing record before proceeding:

- **Cron:** `checkCronApproval()` skips the org with reason 'Fees not yet approved'
- **Manual renewal:** `checkApprovalRequired()` returns 400 error
- **Member portal:** `checkMemberFeesApproval()` blocks payment actions and returns an approval-pending response
- **Token page:** Inline check blocks PO submission and payment with 400 errors

Each check follows the same two-step pattern: 1. Query `system_settings` for `membership_require_approval` — if not 'true', approval is not required (return early) 2. Query `organisation_membership_invoicing.fees_app` for the org/year — if true, proceed; if false, block

Invoicing Mode Guard (Workflow)

The workflow `create_membership` action checks the invoicing mode before creating records: - Queries `organisation_membership_invoicing` for the target year - Falls back to legacy null

membership_year rows - Defaults to 'automatic' if no setting exists - Skips for manual and scheduled modes with descriptive messages

Frontend UI — Organisation Membership Tab

File: client/src/components/OrgMembershipTab.jsx

Layout

The tab shows a two-year rolling view:

1. **Current Year Card** — the active membership year
2. **Next Year Card** — preview of the upcoming year

Each year section uses the shared `YearCostSection` component.

YearCostSection Component

Each year card displays:

- **Cost Breakdown:** Tier label, annual cost, custom discounts, pro-rata, free period, spillover, and final cost
- **Override Controls:** Button to add/edit an override (structure, price, or discount type)
- **Simulate Button:** Runs a dry-run simulation and shows the step-by-step log
- **Invoicing Controls:** Radio buttons for Automatic/Scheduled/Manual, date picker for scheduled, Save button
- **Renew & Invoice Now Button:** Only visible on the current year section when invoicing mode is Manual

Key Frontend Behaviours

- The **Renew & Invoice Now** button passes the current year label to the backend: `manualRenewalMutation.mutate({ membershipYear: currentYearCost?.membershipYear })`
- The next year section passes `onManualRenewal={null}`, which causes the button to not render
- Invoicing mode changes are saved independently per year
- The simulate button calls the backend with the year label and displays detailed steps in a panel
- Override modal supports all three override types with configuration-specific forms
- When a **fee has been recorded** for the current year (`currentYearRecorded` is truthy), the entire invoicing section (radio buttons, date picker, Save, Renew & Invoice Now) is hidden and replaced with a status message. The next year card is unaffected since it always passes `currentYearRecorded={null}`

Mutations

Mutation	Endpoint	Method	Purpose
invoicingMutation	/api/membership/org-membership-invoicing	PUT	Save invoicing mode/date
manualRenewalMutation	/api/membership/org-membership-invoicing	POST	Trigger manual renewal
simulateRenewalMutation	/api/membership/org-membership-simulate	POST	Run dry-run simulation
recordMutation	/api/membership/org-membership	POST	Record fee — saves the simulated cost as a history record without invoicing
removeOverrideMutation	/api/membership/org-membership-override	DELETE	Remove an override
approvalMutation	/api/membership/org-membership-invoicing	PATCH	Approve or unapprove fees for a year

Cache Invalidation

After any mutation, the following query keys are invalidated: - ['org-membership', organizationId] — cost data - ['org-membership-invoicing', organizationId] — invoicing settings - ['org-notes'] — organisation notes (since renewals add notes)

Purchase Order Numbers

Overview

Admins can attach a purchase order (PO) number to an organisation's membership year. The PO number flows through to Xero invoices as the reference field, formatted as **Membership YYYY - PO: XXXXX**. Members can also submit PO numbers via the public token page or the portal membership fees page.

Storage

PO numbers are stored in the `organisation_membership_invoicing` table via the `purchase_order_number` column, scoped by `tenant_id`, `organization_id`, and `membership_year`. The `po_source` column tracks who submitted the PO:

- 'member' — submitted by a member via the public token page or portal
- null — entered or overridden by an admin

Flow

1. Admin enters a PO number in the year card invoicing controls
2. Saved alongside invoicing mode via PUT /api/membership/org-membership-invoicing
3. All three renewal paths (manual, cron, workflow) read the PO from invoicing settings
4. PO is passed to `createXeroMembershipInvoice` as the `reference` field
5. PO is stored in the `organisation_membership_history` record

Member-Submitted PO Numbers

When a member submits a PO number (via token page or portal):

1. The PO is saved to `organisation_membership_invoicing` with `po_source = 'member'`
2. In the admin UI (Organisation Membership Tab), the PO field appears **locked** with a “Supplied by member” label
3. An **unlock button** allows the admin to edit the member-submitted PO
4. When the admin saves (after unlocking and editing), `po_source` is cleared to `null`, removing the lock on subsequent loads
5. If a member later resubmits a PO, `po_source` is set back to `'member'` and the lock reappears

Admin Override of Member PO

The lock/unlock pattern ensures admins can clearly see when a PO was submitted by a member vs. entered by an admin. The admin can always override a member-submitted PO by:

1. Clicking the unlock button next to the locked PO field
2. Editing the PO value
3. Clicking Save — this clears `po_source` to `null`

Key Files

File	Purpose
<code>api/membership/org-membership-invoicing.js</code>	Saves/loads PO alongside invoicing mode; clears <code>po_source</code> on admin save
<code>api/cron/process-membership-renewals.js</code>	Reads PO for cron-generated invoices
<code>api/_lib/workflows.js</code>	Reads PO for workflow-generated invoices
<code>api/public/membership-fees/[token].js</code>	Saves member-submitted PO with <code>po_source: 'member'</code>
<code>api/membership/member-fees.js</code>	Saves member-submitted PO with <code>po_source: 'member'</code> (portal path)

Email Fees & Member Payment Portal

Overview

The “Email Fees” feature allows admins to send a branded email to an organisation’s finance contact, containing a cost breakdown and a link to a public payment page. From the public page, the member can:

- View the full cost breakdown for the membership year
- Submit a purchase order number
- Pay immediately via Stripe (if enabled for the tenant)

Token System

Each “Email Fees” action generates a secure token stored in `membership_fee_tokens`:

Column	Purpose
<code>token</code>	Random UUID, used as the public URL path
<code>status</code>	<code>pending</code> → <code>po_submitted</code> → <code>paid</code> (or <code>expired/cancelled</code>)
<code>expires_at</code>	30 days from creation
<code>stripe_payment_intent_id</code>	Set when card payment is initiated
<code>paid_at</code>	Timestamp when payment confirmed
<code>po_number</code>	PO submitted by the member

Admin Flow

1. Admin clicks “Email Fees” on the year card
2. Modal shows the org’s primary contact email (editable)
3. On send, the backend:
 - Creates a token record
 - Runs simulation for cost breakdown
 - Sends a branded email via tenant’s email service
4. Email contains cost summary and a link to `/membership-fees/{token}`

Member Flow (Public Page)

The public page at `/membership-fees/:token` shows:

1. Tenant branding (logo, primary colour)
2. Organisation name, period, tier
3. Full cost breakdown (gross, discounts, pro-rata, free period)
4. Total amount due
5. PO number input (optional)
6. “Pay Now” button (if Stripe enabled)

Stripe Payment

When the member clicks “Pay Now”:

1. Frontend calls `POST /api/public/membership-fees/:token` with `action: create_payment`
2. Backend creates a Stripe `PaymentIntent` for the total amount
3. Stripe Elements mounts in the page
4. Member enters card details and confirms
5. On success, frontend calls `action: confirm_payment`
6. Backend:
 - Verifies the `PaymentIntent` status is `succeeded` and the amount matches
 - Creates `organisation_membership_history` record with `status: 'active'`, `payment_method: 'stripe'`, and the `Stripe payment_intent_id`
 - Creates Xero invoice with “(PAID)” in the reference
 - Updates token status to `paid`
 - Creates an organisation note with payment details

Stripe Payment as Invoicing Override

A successful Stripe payment **immediately records the membership as paid**, which effectively overrides whatever invoicing mode (Automatic/Scheduled/Manual) the admin has configured for that year. Once the history record exists:

- The **cron job** will find `simResult.existingRecord` and skip the organisation entirely (for automatic mode) or skip record creation (for scheduled mode)
- The **workflow action** will find the existing record and skip
- The **manual renewal** button will show “already exists” if clicked

This means a member paying via Stripe settles the membership independently of the admin’s invoicing settings. The only admin setting that flows through is the **PO number** — if one was saved in the invoicing settings before the member pays, it is included on the history record.

Stripe Minimum Amounts

Stripe enforces minimum payment amounts by currency. The system enforces these both on the frontend (disabling the Pay button) and the backend (returning a 400 error):

Currency	Minimum
GBP	0.30
USD, EUR, AUD, NZD	0.50

Key Files

File	Purpose
<code>api/admin/init-membership-fee-token.js</code>	Database table creation
<code>api/membership/email-fees.js</code>	Email Fees endpoint — token creation, email sending
<code>api/public/membership-fees/[token].js</code>	Public API — token validation, PO submission, payment
<code>client/src/pages/MembershipFeePage.jsx</code>	Public member-facing payment page (token-based)
<code>client/src/components/OrgMembershipTable.jsx</code>	Table with Email Fees button and modal

Portal Membership Fees Page

Overview

In addition to the token-based public page for external contacts, a portal page at `/MembershipFees` is available for authenticated members with the `commerce.membership` role access key. This serves members who have portal access (e.g., a finance person at the organisation) and can view fees and make payments at any time without needing an email link.

Access Control

The page is gated by `commerce.membership` in the role access configuration:

- **commerce.membership** — Controls visibility of the Membership Fees page in the portal navigation

- **commerce.membership.submit-po** — Controls whether the member can submit a purchase order number
- **commerce.membership.pay-online** — Controls whether the member can pay via Stripe

Admins manage these permissions via Role Management / Role Access Config Management.

How It Works

1. Member navigates to `/MembershipFees` in the portal
2. The page calls `GET /api/membership/member-fees` which:
 - Identifies the member from their session
 - Looks up their organisation
 - Runs `simulateMembershipForOrg()` to get current fees
 - Returns the same cost breakdown, PO status, and Stripe availability
3. Member can submit a PO number (stored in `organisation_membership_invoicing` with `po_source: 'member'`)
4. Member can pay via Stripe — same flow as the public page:
 - POST with `action: create_payment` creates a `PaymentIntent`
 - Stripe Elements captures card details
 - POST with `action: confirm_payment` validates payment, then:
 - Creates `organisation_membership_history` record with `status: 'active'`, `payment_method: 'stripe'`, and the Stripe `payment_intent_id`
 - Creates Xero invoice with “(PAID)” in the reference
 - Creates an organisation note with payment details
 - This immediately marks the membership as paid, preventing the cron job from re-invoicing (see Stripe Payment as Invoicing Override)

Difference from Token-Based Page

Aspect	Token Page (<code>/membership-fees/:token</code>)	Portal Page (<code>/MembershipFees</code>)
Authentication	Token in URL	Member session
Access	Anyone with the link	Portal members with role access
Expiry	30-day token expiry	Always available
Use case	External finance contacts	Members with portal login
Branding	Full tenant branding (standalone)	Portal layout (within portal navigation)

Key Files

File	Purpose
<code>api/membership/member-fees.js</code>	Authenticated API — session-based fee lookup, PO, payment
<code>client/src/pages/MembershipFees.js</code>	Portal page component
<code>client/src/lib/roleAccessMap.ts</code>	Role access config entry (<code>commerce.membership</code>)
<code>client/src/pages/pageRegistry.js</code>	Page registered as <code>MembershipFees</code>

Membership Settings

Overview

The Membership Settings page (`/MembershipSettings`) provides tenant-level controls that affect how membership fees are processed and presented to members. Three settings are managed from this page:

1. **Require Fee Approval** — enables the fee approval workflow (see Fee Approval Workflow)
2. **Allow Stripe Payments** — controls whether Stripe payment is offered on member-facing fee pages
3. **Custom Approval Message** — a free-text message shown to members when their fees are pending approval

These settings are stored in the `system_settings` table as key-value pairs scoped by `tenant_id`.

Admin UI

File: `client/src/pages/MembershipSettings.jsx`

The page is accessible from the Portal Menu Management dropdown (registered in the admin navigation). It displays three controls:

- **Require Fee Approval** toggle — when enabled, fees for each org/year must be explicitly approved by an admin before any processing (cron, manual renewal, member payment) can proceed
- **Allow Stripe Payments** toggle — when disabled, the “Pay Now” button and Stripe Elements are hidden from both the portal membership fees page and the public token-based payment page. Defaults to enabled
- **Custom Message** text area — optional message displayed to members on fee pages when their fees are pending approval. If empty, a default message is shown: “Your membership fees are currently being reviewed. You will be notified when they are ready for payment.”

API

File: `api/membership/membership-settings.js`

Method	Purpose
GET	Returns the three settings as a normalised object: <code>{ require_approval, stripe_enabled, custom_message }</code>
PUT	Saves all three settings using upsert (<code>ON CONFLICT tenant_id, setting_key</code>)

GET response defaults:

Field	Default (when no setting exists)
<code>require_approval</code>	<code>false</code>
<code>stripe_enabled</code>	<code>true</code>

Field	Default (when no setting exists)
<code>custom_message</code>	<code>''</code> (empty string)

PUT body:

```
{
  require_approval: true,      // boolean
  stripe_enabled: true,        // boolean
  custom_message: "...",       // string or empty
}
```

Stripe Enablement Control

The `membership_stripe_enabled` setting is checked in two places to control Stripe payment visibility:

1. **Authenticated portal** (`api/membership/member-fees.js`) — the GET response includes `stripeEnabled` based on the setting. The frontend hides the Stripe payment card when `stripeEnabled` is false.
2. **Public token page** (`api/public/membership-fees/[token].js`) — same check on GET. The frontend hides the payment section accordingly.

Important: This setting does not prevent Stripe payments at the API level — it controls frontend visibility only. The approval workflow (if enabled) provides the backend enforcement layer that blocks payment processing.

Key Files

File	Purpose
<code>api/membership/membership-settings.js</code>	Settings API — GET/PUT for all three settings
<code>client/src/pages/MembershipSettingsAdmin</code>	Admin settings page
<code>api/membership/member-fees.js</code>	Reads <code>membership_stripe_enabled</code> for portal fee page
<code>api/public/membership-fees/[token].js</code>	Reads <code>membership_stripe_enabled</code> for public fee page

Fee Approval Workflow

Overview

The fee approval workflow adds an admin-controlled gate to the membership fee lifecycle. When enabled (via `membership_require_approval` in Membership Settings), fees for each organisation/year must be explicitly approved by an admin before any processing path — cron job, manual renewal, member payment, or PO submission — can proceed.

This gives admins the ability to review and verify calculated fees before they are sent to members or processed into invoices.

How It Works

1. Admin enables “Require Fee Approval” in Membership Settings
2. On the Organisation Membership tab, each year card now shows an **Approve Fees** button (next to the Save button in the invoicing controls area)
3. Admin clicks **Approve Fees** to mark that org/year as approved
4. Once approved:
 - The year card gets a green border and “Approved” badge
 - The “Renew & Invoice Now” and “Email Fees” buttons become enabled
 - The cron job will process the org
 - Members can pay or submit POs on the fee pages
5. Admin can click **Unapprove** to revoke approval at any time

Approval State Storage

The approval flag is stored as `fees_approved` (boolean, default `false`) on the `organisation_membership_invoicing` table, scoped by `tenant_id`, `organization_id`, and `membership_year`.

PATCH endpoint (`api/membership/org-membership-invoicing.js`):

```
PATCH /api/membership/org-membership-invoicing
{
  organizationId: "...",
  membershipYear: "2025/2026",
  action: "approve"    // or "unapprove"
}
```

If no invoicing record exists for the org/year, the PATCH handler creates one with `invoicing_mode`: 'manual' and the requested `fees_approved` value. If a record exists, only `fees_approved` and `updated_at` are updated.

Enforcement Points

The approval check follows the same pattern across all four processing paths: query `system_settings` for `membership_require_approval`, and if enabled, query the org/year's `fees_approved` flag.

Path 1: Cron Job File: `api/cron/process-membership-renewals.js` → `checkCronApproval()`

- Called after simulation, before record creation
- If approval required and not approved → org is skipped with reason 'Fees not yet approved'
- Logged in the cron results as a skipped entry

Path 2: Manual Renewal File: `api/membership/org-membership-invoicing.js` → `checkApprovalRequired()`

- Called after simulation and duplicate check, before record insertion
- If approval required and not approved → returns 400 error: 'Fees must be approved before renewal can be processed. Use the Approve Fees button first.'

Path 3: Portal Member Fees (Authenticated) File: `api/membership/member-fees.js` → `checkMemberFeesApproval()`

- **GET:** Returns `approvalPending` and `approvalMessage` in the response. The frontend shows the approval message card and hides payment/PO options.
- **POST** (`create_payment`, `confirm_payment`): Checks approval before processing. Returns 400 error if not approved.

Path 4: Public Token Page File: `api/public/membership-fees/[token].js`

- **GET:** Returns `approvalPending` and `approvalMessage` inline. The frontend shows the approval message and hides payment/PO options.
- **POST** (`submit_po`): Checks approval before saving PO. Returns 400 error if not approved.
- **POST** (`create_payment`, `confirm_payment`): Checks approval before processing. Returns 400 error if not approved.

Frontend Behaviour

Organisation Membership Tab (`OrgMembershipTab.jsx`) When `membership_require_approval` is enabled:

- Each `YearCostSection` receives `approvalRequired`, `feesApproved`, `onApprove`, `onUnapprove`, and `approvePending` props
- **Not approved state:**
 - “Approve Fees” button (primary variant) appears next to Save
 - “Renew & Invoice Now” and “Email Fees” buttons are disabled
 - An amber warning message appears: “Fees must be approved before invoicing or payment actions”
- **Approved state:**
 - Year card gets a green border and light green background (`border-green-200 bg-green-50/50`, dark mode: `border-green-900 bg-green-950/30`)
 - “Approved” badge appears next to the year label
 - “Unapprove” button (outline variant) replaces the “Approve Fees” button
 - All action buttons are enabled
- Approval state is tracked per year in a `feesApprovedMap` keyed by year label
- Both current year and next year cards independently support approve/unapprove

Portal Membership Fees Page (`MembershipFees.jsx`) When `approvalPending` is true in the API response:

- A card with an amber shield icon displays the approval message
- The message is the custom message from Membership Settings, or a default: “Your membership fees are currently being reviewed. You will be notified when they are ready for payment.”
- PO submission and Stripe payment sections are hidden

Public Token Page (`MembershipFeePage.jsx`) The public token page receives `approvalPending` and `approvalMessage` in the GET response from the backend, but the current frontend component does not display a dedicated approval message card. Enforcement is handled at the backend level — POST requests for PO submission and Stripe payment return 400 errors when fees are not approved. The `approvalPending` flag is available in the API response for future frontend use.

Data Flow Diagram

Admin enables "Require Fee Approval" in Membership Settings

- system_settings: membership_require_approval = 'true'

Admin opens Organisation Membership tab

- Year card shows "Approve Fees" button
- Admin reviews calculated fees
- Admin clicks "Approve Fees"
- PATCH /api/membership/org-membership-invoicing
 - Sets fees_approved = true on invoicing record
 - Year card turns green, "Approved" badge appears

Processing paths check approval:

- Cron job: checkCronApproval() → approved? proceed / skip
- Manual renewal: checkApprovalRequired() → approved? proceed / 400 error
- Member portal: checkMemberFeesApproval() → approved? show payment / show message
- Token page: inline check → approved? show payment / show message

Key Files

File	Purpose
api/membership/membership-settings.js	Manages membership_require_approval setting
api/membership/org-membership-invoicing.js	PATCH handler for approve/unapprove; checkApprovalRequired() for manual renewal
api/cron/process-membership-renewal.js	checkCronApproval() for cron path
api/membership/member-fees.js	checkMemberFeesApproval() for portal path
api/public/membership-fees/[token].js	inline approval checks for public token path
client/src/components/OrgMembershipTab.jsx	Unapprove buttons, green card styling, disabled state logic
client/src/pages/MembershipFees.js	Approval-pending message display for portal
client/src/pages/MembershipFeePage.jsx	Approval-pending message display for public page

Database Tables

membership_tier_config

The tier configuration for a tenant. Multiple configs can be active simultaneously if scoped to different organisation field values.

Column	Type	Description
id	uuid	Primary key
tenant_id	uuid	Tenant scope
name	text	Config name
field_source	text	'core' or 'custom'

Column	Type	Description
field_name	text	Field used for band matching (e.g. 'member_count')
field_id	uuid	Custom field ID (if field_source is custom)
membership_start_month	int	Month the membership year begins
membership_start_day	int	Day the membership year begins
currency	text	e.g. 'GBP'
billing_period	text	e.g. 'annual'
prorata_enabled	boolean	Whether pro-rata calculations apply
free_period_amount	int	Free period duration
free_period_unit	text	'months', 'weeks', or 'days'
rollover_enabled	boolean	Whether free period spillover applies
effective_from	date	When this config became active
effective_to	date	Null if currently active
structure_field_id	uuid	Field used for config scoping (optional)
structure_match_value	text	Value to match for scoped configs (optional)
pricing_model	text	'tiered' or 'flat'
start_mode	text	'fixed_date' or 'immediate'
flat_cost	numeric	Cost for flat pricing model

membership_tier_band

Tier bands within a configuration.

Column	Type	Description
id	uuid	Primary key
config_id	uuid	FK to membership_tier_config
tenant_id	uuid	Tenant scope
label	text	Display label (e.g. 'Small (1-10)')
min_value	numeric	Lower bound (inclusive)
max_value	numeric	Upper bound (inclusive), null = unlimited
annual_cost	numeric	Annual fee for this band
vat_rate	text	VAT/tax configuration (JSON string or tax type)

organisation_membership_history

Records of membership renewals — one row per org per year.

Column	Type	Description
id	uuid	Primary key
tenant_id	uuid	Tenant scope
organization_id	uuid	FK to organisation
membership_year	text	Year label (e.g. '2025/2026')
config_id	uuid	Config used for calculation
band_id	uuid	Band matched

Column	Type	Description
tier_label	text	Band label at time of creation
field_value	numeric	Org's field value at time of creation
annual_cost	numeric	Base annual cost
prorata_cost	numeric	Pro-rata cost (null if not applicable)
free_period_discount	numeric	Free period discount amount
rollover_discount	numeric	Rollover/spillover discount
custom_discount_total	numeric	Total custom discounts
custom_discount_details	jsonb	Array of applied discount details
final_cost	numeric	Amount charged
currency	text	Currency code
billing_period	text	e.g. 'annual'
vat_rate	text	VAT configuration snapshot
purchase_order_number	text	PO number (from invoicing settings, member submission, or token page)
payment_method	text	How payment was made: 'stripe', 'xero', or null (admin-initiated)
stripe_payment_intent_id	text	Stripe PaymentIntent ID (if paid via Stripe)
status	text	'active'
notes	text	How the record was created
xero_invoice_id	text	Linked Xero invoice ID
xero_invoice_number	text	Linked Xero invoice number

organisation_membership_invoicing

Per-org, per-year invoicing mode settings.

Column	Type	Description
id	uuid	Primary key
tenant_id	uuid	Tenant scope
organization_id	uuid	FK to organisation
membership_year	text	Year label (null for legacy fallback)
invoicing_mode	text	'automatic', 'scheduled', or 'manual'
invoice_date	date	Scheduled invoice date (for scheduled mode)
purchase_order_number	text	PO number for this year's membership
po_source	text	'member' if PO was submitted by a member (via token page or portal), null if entered by admin
fees_approved	boolean	Whether fees have been approved by an admin for this org/year (default false)
updated_at	timestamp	Last modification

organisation_membership_override

Per-org overrides.

Column	Type	Description
id	uuid	Primary key
tenant_id	uuid	Tenant scope
organization_id	uuid	FK to organisation
membership_year	text	Year label (null for legacy fallback)
override_type	text	'structure', 'price', or 'discount'
config_id	uuid	Alt config (for structure override)
band_id	uuid	Forced band (for structure override)
manual_price	numeric	Fixed price (for price override)
discount_type	text	'percentage' or 'fixed' (for discount override)
discount_value	numeric	Discount amount (for discount override)
note	text	Admin note explaining the override

system_settings

Tenant-scoped key-value store for membership configuration settings. Used by the Membership Settings admin page to control approval workflow, Stripe enablement, and custom member-facing messages.

Column	Type	Description
id	uuid	Primary key
tenant_id	uuid	Tenant scope
setting_key	text	Setting identifier (e.g. 'membership_require_approval')
setting_value	text	Setting value (stored as string)

Membership-related keys:

Key	Values	Default	Description
membership_require_approval	'true' or 'false'	'false' (not present)	When 'true', fees must be approved per org/year before any processing
membership_stripe_enabled	'true' or 'false'	'true' (not present)	When 'false', Stripe payment is hidden on member-facing pages
membership_custom_message	'none' or 'message'	'none' (not present)	Custom message shown to members when fees are pending approval

Data Flow Diagrams

Automatic Renewal Flow

Go-live date set on org

- Workflow triggered
 - create_membership action
 - simulateMembershipForOrg(source: 'workflow')
 - Check invoicing mode = 'automatic'
 - Check no existing record
 - Insert membership_history record (no invoice)
 - Return success

Daily cron runs

- Find orgs with 'automatic' invoicing
- For each org:
 - simulateMembershipForOrg(source: 'cron')
 - Go-live date set?
 - Year started?
 - Approval required? → Check fees_approved (skip if not approved)
 - Existing record? → Skip entirely (record + invoice already handled or invoice not needed)
 - No existing record? → Create record + invoice together

Note on Automatic mode invoice timing: In the typical flow, the workflow creates the record when the go-live date is set. The cron job then finds the existing record and skips. This means the invoice is generated by the cron only when it creates the record itself (i.e. when the workflow didn't run or was missed). If you need the workflow to trigger immediate invoicing for automatic mode, that would require a separate enhancement. Currently, for most automatic orgs, the cron job is the primary path that creates both the record and invoice together.

Scheduled Renewal Flow

Go-live date set on org

- Workflow triggered
 - create_membership action
 - simulateMembershipForOrg(source: 'workflow')
 - Check invoicing mode = 'scheduled'
 - SKIP: "Scheduled renewal job will handle this"

Daily cron runs (before invoice date)

- Find orgs with 'scheduled' invoicing
- Year started?
- No existing record? → Create record WITHOUT invoice
- Invoice date reached? → Skip invoicing

Daily cron runs (on/after invoice date)

- Find orgs with 'scheduled' invoicing
- Existing record WITHOUT invoice?
- Invoice date reached?
- Generate Xero invoice from stored record data
- Link invoice to history record

Manual Renewal Flow

Go-live date set on org

- Workflow triggered
- create_membership action
 - simulateMembershipForOrg(source: 'workflow')
 - Check invoicing mode = 'manual'
 - SKIP: "Use admin UI Renew & Invoice Now button"

Admin opens Organisation Membership tab

- Views current year cost breakdown
 - Clicks "Renew & Invoice Now"
 - POST /api/membership/org-membership-invoicing
 - simulateMembershipForOrg(source: 'manual', targetYear: currentYear)
 - Go-live date set?
 - No existing record?
 - Approval required? → Check fees_approved (400 error if not approved)
 - Insert membership_history record
 - Create Xero invoice immediately
 - Add organisation note
 - Return success
-

Xero Integration

Invoice Creation

When a renewal creates an invoice, it calls `createXeroMembershipInvoice()` with:

Field	Source
appTenantId	Tenant ID (for Xero connection lookup)
organizationName	Organisation name (invoice contact)
membershipYear	Year label (for reference)
tierLabel	Matched band label
finalCost	Calculated final cost
currency	From tier config
reference	"Membership {yearLabel}"
vatRate	From matched band's vat_rate

Xero Settings

Two tenant-level settings control invoice behaviour:

- `xero_sales_account_code`: Account code for the line item (default: '200')
- `xero_invoice_status`: Status of created invoices (default: 'DRAFT')

Invoice Linking

After creating an invoice, the `xero_invoice_id` and `xero_invoice_number` are stored on the history record for tracking and to prevent duplicate invoicing.

VAT Handling for Scheduled Invoicing

When the cron job invoices an existing record (scheduled mode), it derives VAT from: 1. The stored record's `vat_rate` field (if populated at record creation time) 2. Falling back to looking up the band's `vat_rate` by `record.band_id`

This ensures the invoice matches the recorded cost, even if tier bands have changed since the record was created.

Configuration Reference

Tier Configuration Wizard

The tier management UI uses a 6-step wizard:

1. **Structure Scope:** Optional scoping by organisation field value
2. **Tier Model:** Tiered (band-based) or flat cost
3. **Period:** Fixed date (annual from start date) or immediate start
4. **Discounts:** Custom discount rules based on organisation fields
5. **Pricing:** Band definitions with costs and VAT
6. **Summary:** Review and save

Go-Live Date Field

The go-live date is stored as an organisation custom field with `name = 'go_live'`. The simulation looks up this field by:

1. Finding the `preference_field` record with `name = 'go_live'` and `entity_scope = 'organization'` for the tenant
2. Querying `organization_preference_value` for that field and organisation

This field is typically set by a workflow action (e.g. as part of a Due Diligence completion workflow).

Free Period Configuration

Field	Description
<code>free_period_amount</code>	Number of units
<code>free_period_unit</code>	'months', 'weeks', or 'days'

Internally converted to days: $\text{months} \times 30.44$, $\text{weeks} \times 7$.

Rollover

When `rollover_enabled` is true and a free period is configured, any unused free days from year 1 are carried forward as a discount in year 2.

Troubleshooting

Organisation doesn't match any tier band

Symptom: Simulation returns “Organisation does not match any tier band”

Check: - Does the organisation have a value set for the field used by the tier config? - Is the value within the range of any defined band? - If using scoped configs, does the org match the scope criteria?

Renewal was skipped unexpectedly

Check the invoicing mode: If set to manual or scheduled, the workflow action will skip.

Check the go-live date: If not set, automatic renewals are skipped.

Check for existing record: If a record already exists for that year, all paths skip.

Check fee approval: If `membership_require_approval` is enabled and `fees_approved` is false for the org/year, all processing paths (cron, manual, member payment) will be blocked.

Use the **Simulate** button in the UI to see the step-by-step breakdown.

Member cannot pay or submit PO

Symptom: Member sees an approval-pending message instead of payment/PO options on the membership fees page (portal or token-based).

Cause: The tenant has `membership_require_approval` enabled, and the admin has not yet approved fees for that organisation/year.

Fix: Admin navigates to the organisation's Membership tab and clicks the “Approve Fees” button on the relevant year card. Alternatively, disable the approval requirement in Membership Settings if it's no longer needed.

Stripe payment option not showing for members

Symptom: Members see the fee breakdown but no “Pay Now” button on the portal or token-based payment page.

Cause: The tenant has `membership_stripe_enabled` set to 'false' in Membership Settings.

Fix: Admin navigates to Membership Settings and enables the “Allow Stripe Payments” toggle.

Duplicate record error

If you see error code 23505, this means the database constraint caught a race condition. The record already exists — this is the safeguard working correctly. No action needed.

Scheduled invoice not generating

Check: - Is the invoice date set on the year's invoicing settings? - Has the invoice date been reached? - Does the membership history record exist but lack an `xero_invoice_id`? - Is the cron job running?

VAT mismatch on scheduled invoices

Scheduled invoices use VAT from the stored record/band, not from re-simulation. If the tier band's VAT changed between record creation and invoice date, the original rate is used (by design — the invoice should match the recorded amount).

Cost showing differently in simulation vs. actual record

The simulation and record creation use the exact same function (`simulateMembershipForOrg`). If values differ, check: - Was an override added or removed between simulation and creation? - Did the organisation's field value change? - Did the tier config change?

The simulation is a point-in-time calculation — it reflects the state at the moment it runs.